

TCN: An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling

📅 작성일시	@2025년 3월 18일
📍 강의 번호	BASE
📄 유형	논문리뷰
📎 자료	https://arxiv.org/pdf/1803.01271
☑️ 복습	☐

논문 핵심 요약

현재까지 거의 대부분의 딥러닝 사용자들에게 sequence modeling은 RNN 처럼 여겨져왔음.

그러나, 최근 연구 결과는 Convolutional Neural Network (CNN) 기반 구조(WaveNet, Gated ConvNet emd)으로 오디오 합성, 기계 번역, 언어 모델링 같은 분야에서 RNN보다 더 좋은 성능을 보여주었음.

이 논문에서 다양한 표준 벤치마크(음악 모델링, 언어 모델링, 합성 테스트 등)에 대해, "RNN(대표적 LSTM, GRU)"과 "일반적인 구조의 CNN(논문에서 TCN이라 부르는 구조)"을 시스템적으로 비교 및 평가 함.

결과적으로 TCN (Temporal Convolutional Network) 구조가 전통적인 RNN보다 더 좋은 성능을 내는 경우가 많았고, 특히 더 긴 의존관계(장기메모리)가 필요한 문제에서 더 우수함을 보여줌.

이러한 결과를 통해 "시퀀스 모델링 = RNN"이라는 기존의 통념을 재고할 필요가 있으며, CNN방식을 처음부터 고려해보는 것이 효과적일 수 있음을 주장함.

Abstract

많은 사람들이 "순차적 데이터 처리"라고 하면 자연스럽게 RNN을 떠올리는데, 최근 연구들에서 CNN기반 방식이 기계 번역이나 오디오 합성 등 특정 문제에서는 RNN보다 좋다는 결과가 나오고 있음. 그래서 이 논문에서는 "여러 가지 시퀀스 모델링 문제에서 과연 어떤 아키텍처를 쓰는 것이 좋은가?"라는 질문을 던짐

CNN과 RNN을 같은 기준에서 체계적으로 비교 및 평가함.

음악 모델링, 언어 모델링, 합성 테스트, 이미지 등 다양한 과제를 실험 벤치마크로 삼음. TCN이라 불리는 단순하지만 효과적인 CNN아키텍처가, 전통적인 LSTM이나 GRU보다 다양한 작업에서 더 낫다는 결론.

특히 TCN이 장기 의존성을 더 잘 학습하는 특성을 보였음. (RNN은 이론적으로는 무한 기억이 가능하지만 실제로는 잘 안되는 경우가 많았다는 분석)

최종적으로 "시퀀스 모델링 할 때 RNN만을 고집할 필요가 없다. CNN 접근법이 더 쉽고 성능도 좋으며, 적은 튜닝으로도 강력하다"는 것.

Introduction

딥러닝 교과서나 강의들을 보면, 순차적 데이터(시계열)을 다루는 방법으로 대부분 RNN을 소개.

하지만, CNN을 이용한 오디오 합성, 기계 번역(ConvS2S), 언어모델링(Gated CNN)등에서 "CNN이 RNN보다 더 좋은 결과"를 보고하는 사례가 늘어나고 있음.

"이것들이 특정 과제(도메인)에만 통하는 건지, 아니면 전반적인 시퀀스 모델링 문제에도 적용되는건지?" 궁금 RNN과 CNN 두 가지 접근을, 여러 가지 표준 시퀀스 모델링 벤치마크에 걸쳐 체계적으로 비교해보자. 특히 기존 논문들에서 RNN이 잘한다고 알려진 "대표 과제"들에 CNN을 그대로 대입해 성능을 확인.

CNN쪽은 TCN이라는 단순 구조를 공통적으로 사용.

RNN(LSTM, GRU, vanilla RNN) 대비 TCN이 더 좋은 결과를 보여줌. 뿐만 아니라, RNN은 이론적으로 무한 길이의 기억을 가질 수 있음에도, 실제로 "기울기 소실(gradient vanishing)"등으로 장기 의존 처리에 취약한 경향이 확인됨.

반면 TCN은 병렬화도 쉽고, dilated convolution과 residual 구조 덕분에 더 긴 과거 정보를 잘 반영하는 것으로 나타남.

따라서 "순차적 모델링 = RNN"이라는 기존 상식을 재검토 해야함. 특히 딥러닝 실무에서 "시퀀스를 처음 다룰 때 TCN(혹은 CNN계열)을 기본 옵션으로 고려하는게 어떨까?"라고 제안함.

Background

전통적인 시퀀스 모델링과 RNN

RNN은 과거의 은닉 상태를 다음 시점으로 전달함으로써, 자연스럽게 순차 정보를 다룰 수 있음. LSTM, GRU 등 다양한 RNN 변형들이 제안되었고, 언어모델링이나 음성 처리 등에서 큰 성공을 거둠.

그러나 RNN은 오래된 과거 입력에 대한 "장기 메모리(Long-Term Dependence)" 학습이 어려운 문제가 있었고, 폭발적/소실적 기울기도 자주 언급됨.

CNN을 이용한 시퀀스 처리 연구

사실 CNN을 시퀀스 데이터에 적용하는 아이디어는 예전부터 있었음. 이후 NLP 태깅 및 분류 등에 CNN이 도입되었고, 최근에는 WaveNet, Gated ConvNet, ConvS2S 등에서 **RNN없이도 가능하다, 그리고 종종 더 좋다는** 결과가 나타남.

이 논문의 비교 실험 의의

이전까지는 "RNN 변형을 서로 비교"하는 논문이 많았음. CNN과 RNN을 같은 문제 셋에서 공정하게 비교한 실험은 상대적으로 부족했음. 이 논문은 RNN이 강세를 보인다고 알려진 여러 벤치마크를 대상으로, 단순화된 CNN 구조와 RNN을 정면 비교함.

목표 : "CNN 방식의 성공이 특정 문제에만 적용되는 게 아니라, 일반적인 시퀀스 모델링 전반에 걸쳐 가능성 있는 접근임"을 체계적으로 보이려 함.

Temporal Convolutional Networks

논문에서 제안한 TCN은 과거 CNN연구에서 제안된 여러 기법을 시퀀스 모델링에 적용해서 만든 "간단한 아키텍처"임.

핵심 포인트

1. 순방향(미래→과거) 정보 누출이 없어야 함.(=인과성, Causality 보장)
 - 이를 위해 "미래 시점은 과거 시점 정보만 참조할 수 있는" **"Causal Convolution"**을 사용

2. 입력과 출력의 길이를 동일하게 유지 (Fully Convolutional Network)

Sequence Modeling 관점

시퀀스 모델링의 일반적 목표 : 입력 시퀀스 $\{x_t\}$ 가 있을 때, 각 시점 t 에서 예측해야 할 출력 $\{y_t\}$ 를 잘 맞추는 것. 시간 t 의 출력을 예측할 때는 그 시점까지의 입력 $(x_0, \dots, x_t)$ 만 사용할 수 있어야함. (미래 정보는 사용 불가능) RNN은 구조적으로 이를 만족하지만, CNN을 쓸 때는 특별히 "인과적(미래 정보가 새지 않는)" 설정을 해줘야 함.

Causal Convolutions (인과적 합성곱)

일반적인 1D CNN은 컨볼루션 필터가 입력 전체 (미래 포함)에 적용될 수 있어서, 시계열 예측시 "미래 정보를 미리 봐버리는" 문제가 생길 수 있음.

Causal Convolution은 컨볼루션 필터가 "현재시점 t 와 그 이전 시점 ($\leq t$)"만 사용하도록 제로 패딩(Zero padding) + 마스킹을 적용

이렇게 하면 RNN과 마찬가지로, "과거 정보만 참조하고 미래는 참조불가" 라는 시간적 인과성을 보장

Dilated Convolutions (팽창 합성곱)

시계열 예측에서 멀리 떨어진 과거 정보도 반영해야 할 때, 단순 Causal Conv 만으로는 깊이(레이어)나 필터 크기가 지나치게 커야 함.

Dilated Convolution을 쓰면, 커널을 일정 간격 만큼 건너뛰며 적용함으로써,

- 적은 레이어와 필터 크기로도 지수적으로 넓은 범위의 과거 시점에서 접근 가능.
- 예: dilation factor가 1,2,4,8 등으로 증가하면, 깊이가 조금만 깊어져도 매우 먼 과거까지 한 번에 볼 수 있음 (WaveNet 등에서 사용)

Residual Connections (잔차 연결)

딥 네트워크일수록 기울기 소실 위험이 커지므로, ResNet에서 사용한 "입력 x 에 변환 $F(x)$ 을 더한 뒤 활성화"하는 방식을 채택.

TCN에서도 레이어마다 입력 + $F(\text{입력})$ 을 합치는 residual block을 사용하여,

- 훈련 안정성
- 학습 효율

등을 개선할 수 있음.

단, CNN은 레이어마다 채널 차원이 달라질 수 있어, 필요하면 1x1 합성곱 등을 통해 차원을 맞추고 residual sum을 수행함.

TCN의 장단점 정리

장점

- 병렬화가 쉬움
 - RNN은 순차적으로 hidden state를 업데이트해야 하나, CNN(합성곱)은 시퀀스 전체를 한꺼번에 처리 가능 → 학습 및 추론 속도에 유리
- 유연한 receptive field
 - dilated convolution과 필터 크기를 조절함으로써 멀리 떨어진 과거도 쉽게 참조 가능

- 안정적인 기울기 흐름
 - RNN에서 자주 발생하는 기울기 폭발/ 소실 문제가 상대적으로 적음
- 메모리 사용량
 - RNN은 시퀀스 길이에 따라 hidden state를 timestep 마다 관리해야 하지만, TCN은 네트워크 깊이에 비례해 메모리를 소모하여 상대적으로 안정적.

단점

- 추론 시 (실시간 상황) 메모리
 - RNN은 마지막 hidden state만 남겨두면 과거 정보가 요약되지만, TCN은 필요한 과거 샘플 (혹은 중간계산)을 일정 길이만큼 유지해야 함.
- 도메인 전이 시 파라미터 재설정
 - 실제 시퀀스 길이(필요한 문맥 길이)가 크게 달라지면, dilated conv 세팅 (k나 d)을 다시 조정해야 할 수 있음.

▼ 핵심 정리

- **TCN은 1D CNN을 시퀀스 모델링에 맞춰 인과성(미래 누출 방지)과 dilated convolution, residual block 등을 결합한 구조이다.**
- 기존 RNN이 가진 장단점(이론적 '무한 메모리' vs. 실제 기울기 소실 문제 등)과 달리, TCN은 **병렬화·장기 의존 학습 등에 강점**을 갖는다.
- 다만, "아주 긴 시퀀스에서도 직관적으로 hidden state만 보관하면 되는 RNN"에 비해, TCN은 "과거 입력을 일정 기간(또는 레이어 중간결과) 보관해야 하는" 불편함이 있을 수 있다.

Sequence Modeling Tasks

논문에서는 RNN을 평가할 때 많이 쓰이는 **표준 벤치마크**들을 선택하여, RNN(LSTM, GRU 등)과 논문에서 제안한 **TCN**을 체계적으로 비교한다.

구체적인 과제는 다음과 같다:

1. Adding Problem

- 랜덤 값을 더하는 단순 합산 테스트로, 시점이 길어질수록 먼 과거 정보를 얼마나 잘 유지·활용하는지 확인.

2. Sequential/Permuted MNIST

- MNIST 이미지를 한 줄로 펼쳐(784차원) 시퀀스로 입력.
- 시퀀스 순서를 임의로 섞는 Permuted MNIST는 더 어려움.
- 긴 시퀀스에서 RNN 또는 TCN이 얼마나 잘 학습하는지 확인.

3. Copy Memory Task

- 초반부의 10개 숫자를, (중간에 긴 공백 구간 후) 마지막 구간에서 그대로 재현해야 하는 과제.
- "멀리 떨어진 과거 정보를 잘 기억하고, 정확히 복원해내는가?"를 보는 전형적 장기 의존성 테스트.

4. Polyphonic Music Modeling

- 두 가지 데이터셋: JSB Chorales(바흐 코랄, 382개)와 Nottingham(1200곡의 포크 튠)
- 매 시점마다 연주되는 건반 상태(88개 음높이의 on/off)를 예측.

- 음악 분야에서 RNN이 많이 쓰이던 대표 사례.

5. Language Modeling

- 단어 수준(Word-level): PennTreebank(상대적으로 작은 규모), WikiText-103(대규모), LAMBADA(맥락 길이가 긴 문맥 예측)
- 문자 수준(Character-level): PennTreebank(문자 버전), text8(약 1억 문자 규모)
- 언어 모델링은 RNN의 대표 응용 분야로, 과거 문맥 정보를 얼마나 잘 활용하는지 측정.

Experiments (실험 및 결과)

실험 개요

- 위 7가지 과제(합성 테스트, 음악 모델링, 언어 모델링 등)에 대해,
 - *TCN vs. RNN(LSTM, GRU, vanilla RNN)**을 **동일 파라미터 규모**로 세팅해 비교.
- 특별히 TCN 쪽은 논문에서 제안한 "Dilated Causal Convolution + Residual Block" 구조를 동일하게 적용하고,
 - 하이퍼파라미터(필터 크기, dilation 등)를 적절히 조정해 **충분히 먼 과거**를 볼 수 있게 함.

주요 결과 요약

- Adding Problem, Copy Memory Task 등
 - TCN이 RNN보다 훨씬 빠르게 수렴하고 최종 오차도 매우 낮음.
 - GRU, LSTM은 어느 정도까진 개선되지만, 시퀀스가 길어질수록 기억 성능이 크게 떨어진다.
- Sequential/Permuted MNIST
 - TCN이 RNN 대비 더 높은 정확도를 보임.
 - Permuted MNIST에서도 TCN이 안정적으로 학습하며 SOTA(동 시점 기준)를 상회.
- Polyphonic Music Modeling
 - JSB Chorales, Nottingham 모두 TCN이 LSTM, GRU보다 더 낮은 NLL(negative log-likelihood)을 달성.
 - 소규모 데이터셋이므로 특화된 기법(특수한 RNN 변형 등)도 존재하나, 단순 TCN으로도 RNN보다 우수함을 확인.
- Word/Character-level Language Modeling
 - 소규모 데이터(PTB 단어/문자)에서는 잘 튜닝된 LSTM도 상당히 우수하나, TCN 역시 GRU, vanilla RNN보다 앞서고 비슷하거나 더 나은 성능도 가능.
 - 대규모 데이터(WikiText-103, LAMBADA, text8 등)에서는 TCN이 높은 정확도(낮은 perplexity, 낮은 bpc)를 보이며, 장기 문맥 처리에 강점을 보임.

Memory Retention Analysis

- Copy Memory나 LAMBADA와 같이 긴 문맥이나 먼 과거 정보를 필요로 하는 과제에서, TCN이 RNN 대비 유의미하게 높은 "기억 유지 능력"을 발휘.
 - RNN은 이론적으로 무한 기억이 가능하지만, 실제로는 기울기 소실/폭발로 인해 장기정보를 잘 못 살린다는 점이 재확인됨.

- TCN은 dilated conv와 잔차 연결 덕분에 먼 과거도 효과적으로 커버.

Conclusion

1. 시퀀스 모델링 = RNN 공식에 대한 재고

- 오랜 기간 RNN이 “순차 데이터 처리”의 표준으로 여겨졌지만,
- 이번 실험 결과에서는 TCN 같은 CNN 기반 구조가 매우 경쟁력 있음을 폭넓게 확인.

2. TCN의 실용적 이점

- 병렬화 용이, 안정적인 기울기 흐름, 장기 메모리에 대한 강점 등을 가짐.
- 언어 모델링, 음악 모델링, 합성 테스트 등 다양한 영역에서 단순한 설정만으로 RNN보다 좋은 결과를 달성.

3. 향후 전망

- RNN 쪽(LSTM, GRU)은 수많은 변형과 튜닝 기법(예: dropout, gating, skip connection 등)이 연구되어왔음.
- 반면 TCN 계열은 상대적으로 최적화·정교화 연구가 적은 편이므로, 추가 발전 가능성 큼(더 좋은 정규화, 게이트 기법, skip 연결 등).
- 결론적으로, “향후 시퀀스 모델링 작업을 시작할 때, TCN(CNN 기반)도 매우 유력한 초기 선택지”로 삼길 제안함.